

Trabalho 1 — Implementação do Método Simplex de Duas Fases

Instituto Federal do Norte de Minas Gerais — Campus Montes Claros

Curso	Ciência da Computação — 5º período
Disciplina	Pesquisa Operacional — 2026/2
Professor	Alberto Alexandre Assis Miranda
Valor	(Parte dos 25 pontos da média de tarefas do Bloco 1)
Modalidade	Individual
Linguagem	C ou C++
Entrega	Google Classroom (turma yfcm23og)
Prazo sugerido	Segunda-feira, 21/09/2026, às 23h59 (confirmar a data no Classroom)

1. Objetivo

Implementar, do zero, um resolvidor de Programação Linear baseado no **Método Simplex de Duas Fases**, capaz de:

1. ler um modelo de PL genérico (maximização ou minimização, com restrições $\leq$, $\geq$ e $=$);
2. colocá-lo automaticamente na forma padrão, introduzindo variáveis de folga, de excesso e artificiais;
3. executar a **Fase 1** (minimização da soma das variáveis artificiais) e a **Fase 2** (otimização do objetivo original);
4. reportar a solução ótima **ou** identificar corretamente que o problema é **inviável** ou **ilimitado**;
5. reconhecer, na tabela final, os casos especiais de **múltiplas soluções ótimas** e de **degenerescência**;
6. exibir, sob demanda, o **passo a passo** (tableaus de cada iteração), permitindo conferir manualmente o que o programa fez.

O trabalho consolida o conteúdo das aulas 4 e 5 (Método Simplex; Big-M e Duas Fases; casos especiais) e é pré-requisito conceitual para os temas de dualidade e de Programação Linear Inteira, que virão a seguir.

Por que Duas Fases e não Big-M? Como discutido em aula, o Big-M exige escolher um valor numérico "grande o suficiente" para M, o que em ponto flutuante gera erros de arredondamento e comparações instáveis. O método das Duas Fases não depende de nenhuma constante arbitrária — por isso é o preferido em implementações computacionais. (Uma implementação adicional do Big-M é oferecida como bônus opcional na Seção 12.)

2. O que deve ser implementado

O programa deve ser um executável de linha de comando chamado `simplex`, que:

- **lê o modelo da entrada padrão (`stdin`);**
- **escreve o resultado na saída padrão (`stdout`)** no formato canônico da Seção 5;
- se receber o argumento `-t` (*trace*), imprime **antes** do resultado o passo a passo descrito na Seção 6.

```
./simplex          < entrada.txt      # apenas o resultado
./simplex -t       < entrada.txt      # passo a passo + resultado
```

Todas as decisões do algoritmo são **determinísticas** e estão fixadas na Seção 3. Isso é essencial: a correção automatizada compara a saída do seu programa, caractere a caractere (com tolerância numérica), com a saída de referência. Duas implementações corretas do Simplex podem chegar a vértices ótimos diferentes quando há empates — por isso as regras de desempate abaixo **não são sugestões, são obrigatórias**.

3. Especificação do algoritmo (regras obrigatórias)

3.1 Pré-processamento e padronização

Seja o modelo lido com n variáveis de decisão $x_1..x_n$ (todas com $x_j \geq 0$) e m restrições.

Passo 1 — Lado direito negativo. Se $b_i < 0$ em alguma restrição i , multiplique a **linha inteira** (coeficientes e b_i) por -1 e **inverte o sentido** da restrição: $\leq$ vira $\geq$, $\geq$ vira $\leq$, $=$ permanece $=$. Faça isso antes de qualquer outra coisa.

Passo 2 — Variáveis auxiliares. Para cada restrição i (na ordem em que aparecem na entrada):

Sentido	Transformação	Variáveis criadas
$\leq$	$\dots + s_i = b_i$	folga s_i
$\geq$	$\dots - e_i + a_i = b_i$	excesso e_i e artificial a_i
$=$	$\dots + a_i = b_i$	artificial a_i

Todas as variáveis criadas são ≥ 0 . O índice da variável auxiliar é o **índice da restrição** que a originou (a folga da 3ª restrição chama-se s_3 ; a artificial da 3ª restrição chama-se a_3).

Passo 3 — Ordem das colunas do tableau. Obrigatória, pois define os desempates:

```
x1, x2, ..., xn, [folgas e excessos, na ordem das restrições], [artificiais, na ordem das restrições], RHS
```

Passo 4 — Base inicial. Para cada linha i : a variável básica inicial é a_i , se a restrição gerou artificial; caso contrário, é s_i .

Passo 5 — Sentido da otimização. Trabalhe internamente sempre com **maximização**. Se a entrada for `MIN`, otimize `-c` e, ao imprimir, reporte **Z no sentido original** (isto é, multiplique o valor ótimo interno por -1). O vetor x não muda.

3.2 Regras de pivoteamento (idênticas nas duas fases)

Considere a linha Z do tableau escrita na forma dos **custos reduzidos** (como feito em aula: a linha Z do tableau inicial de $\text{Max } Z = 18x_1 + 25x_2$ é $-18 \ -25 \ \dots$).

- **Variável que entra:** a coluna com o **valor mais negativo** na linha Z . Em caso de empate, escolha a de **menor índice de coluna** (a mais à esquerda, segundo a ordem do Passo 3). Se nenhum valor for negativo (isto é, todos $\geq -1e-9$), a tabela é ótima — pare.
- **Variável que sai:** teste da razão mínima RHS_i / a_{ip} , considerado **apenas** para linhas com $a_{ip} > 1e-9$ (coeficiente **estritamente** positivo na coluna que entra). Escolha a **menor razão**; em caso de empate, a de **menor índice de linha** (a mais acima).
- **Ilimitada:** se nenhuma linha qualificar para o teste da razão, o problema é ilimitado — pare e reporte.
- **Pivoteamento:** divida a linha pivô pelo elemento pivô e zere a coluna pivô em todas as demais linhas **e na linha Z** .

3.3 Fase 1

Só é executada **se houver pelo menos uma variável artificial**. Se todas as restrições forem $\leq$ com $b_i \geq 0$, a Fase 1 é dispensada e o contador de iterações da Fase 1 é 0 .

1. Objetivo da Fase 1: **minimizar $w = \text{soma das artificiais}$** , equivalente a maximizar $-w$.
2. Construa a linha Z da Fase 1 já em forma canônica: comece com 0 em todas as colunas e **subtraia**, dela, cada linha do tableau cuja variável básica seja artificial. Ao final, force 0 nas colunas das próprias artificiais.
3. Itere segundo a Seção 3.2, **proibindo a reentrada de variáveis artificiais na base** (as colunas artificiais nunca são candidatas a entrar).
4. Ao terminar: seja w^* o valor ótimo da Fase 1 (o valor de w , não de $-w$).
 - Se $w^* > 1e-7$: o problema original é **INVIÁVEL**. Encerre imediatamente.
 - Se $w^* \approx 0$: prossiga para a transição.

3.4 Transição Fase 1 → Fase 2

Pode acontecer de uma artificial permanecer básica com valor **zero** ao fim da Fase 1. Trate assim, percorrendo as linhas de cima para baixo:

- Se a linha i tem uma artificial na base e existe alguma coluna **não artificial** j com $|a_{ij}| > 1e-9$: pivoteie em (i, j) , usando o **menor j** disponível. Isso troca a artificial por uma variável legítima sem alterar a solução.
- Se **todos** os coeficientes não artificiais da linha i forem nulos: a restrição é **redundante** (combinação linear das demais). **Remova essa linha** do tableau e siga.

Depois disso, **descarte todas as colunas artificiais**.

3.5 Fase 2

1. Reconstrua a linha Z a partir da função objetivo **original** (na forma de maximização): $Z_j = -c_j$ para as variáveis de decisão e 0 para folgas e excessos.
2. **Canonicalize:** para cada linha i cuja variável básica seja $b(i)$, se a linha Z tiver valor não nulo na coluna $b(i)$, subtraia dela um múltiplo apropriado da linha i , até que toda coluna básica tenha 0 na linha Z .
3. Itere segundo a Seção 3.2 até a otimalidade ou até detectar ilimitação.

3.6 Detecção dos casos especiais (na tabela ótima)

- **Múltiplas soluções ótimas** — `MULTIPLAS: SIM` se existir alguma variável **não básica** (entre variáveis de decisão, folgas e excessos) com custo reduzido **nulo** ($|Z_j| < 1e-7$) na linha `Z` da tabela ótima.
- **Solução alternativa** — quando `MULTIPLAS: SIM`, execute **um pivoteamento adicional** trazendo para a base a **primeira** (menor índice de coluna) variável não básica de custo reduzido nulo, com a saída decidida pelo teste da razão mínima usual. Imprima o novo vetor `X` como `ALTERNATIVA`. Se nenhuma linha qualificar para o teste da razão nessa coluna, a aresta ótima é ilimitada: imprima `ALTERNATIVA: RAI0`.
- **Degenerescência** — `DEGENERADA: SIM` se alguma variável **básica** tiver valor `0` ($|RHS_i| < 1e-7$) na tabela ótima.

Observação conceitual (vale para a discussão do README): sob degenerescência, um custo reduzido nulo em coluna não básica **não garante** um vértice ótimo distinto — o pivoteamento extra pode devolver o mesmo ponto com outra base. O critério algébrico acima é o exigido nesta tarefa, mas comente essa limitação no seu README.

3.7 Cuidados numéricos

- Use `double` em toda a aritmética.
- Adote a tolerância `EPS = 1e-9` para comparações do algoritmo (negatividade da linha `Z`, positividade do pivô) e `1e-7` para as classificações da Seção 3.6.
- **Nunca** compare valores de ponto flutuante com `==`.
- Ao imprimir, valores como `-0.000000` devem sair como `0.000000`.

4. Formato de entrada

A entrada é lida de `stdin`, em texto, com os campos separados por espaços e/ou quebras de linha (o programa **não** deve depender da quebra de linha exata):

```
n m
SENTIDO
c1 c2 ... cn
a11 a12 ... a1n OP1 b1
a21 a22 ... a2n OP2 b2
...
am1 am2 ... amn OPm bm
```

Campo	Descrição
n	número de variáveis de decisão ($1 \leq n \leq 2000$)
m	número de restrições ($1 \leq m \leq 2000$)
SENTIDO	MAX ou MIN (sempre em maiúsculas)
cj	coeficientes da função objetivo (números reais, podem ser negativos)
aij	coeficientes das restrições (números reais)
OPi	um entre $\leq$, $\geq$, $=$
bi	lado direito (número real, pode ser negativo — veja Seção 3.1, Passo 1)

As restrições de não negatividade $x_j \geq 0$ são **implícitas** e não aparecem na entrada. Os números podem ser inteiros ou decimais, com ponto como separador (1.5 , -0.25 , 300).

Sobre os limites de tamanho. A esmagadora maioria dos casos é minúscula ($n, m \leq 10$) e dá para conferir no papel — mas o limite declarado acima é 2000 porque a bateria da Seção 7.1 vai até $n = 1600$, $m = 1920$, justamente para expor problemas de desempenho e de memória que casos pequenos escondem. Duas consequências práticas para a sua implementação:

- **Nada de matriz de tamanho fixo.** Uma declaração como `double T[100][100]` quebra em silêncio (ou estoura a pilha) muito antes desse limite. Aloque o tableau dinamicamente com os valores lidos de n e m — veja a Seção 8, item 4.
- **Dimensione o tableau pelo pior caso.** Se todas as m restrições forem do tipo $\geq$, cada uma cria **duas** colunas (excesso e artificial): a tabela tem m linhas por $n + 2m + 1$ colunas. Para os maiores casos da bateria isso fica na casa de algumas dezenas de MB em `double` — perfeitamente viável, desde que você não faça cópias da tabela inteira dentro do laço de pivoteamento.

Exemplo de entrada (Problema E, visto na aula 5):

```
2 3
MAX
18 25
1.5 2 <= 300
0.5 1 <= 125
1 0 >= 20
```

que corresponde a:

```
Max Z = 18x1 + 25x2
s.a. 1.5x1 + 2x2 <= 300
     0.5x1 + x2 <= 125
       x1      >= 20
     x1, x2 >= 0
```

5. Formato de saída (canônico)

Escreva em `stdout`, exatamente nesta ordem e grafia (sem acentos, para evitar problemas de codificação):

Se a solução ótima existir:

```
STATUS: OTIMA
Z: <valor com 6 casas decimais>
X: <x1> <x2> ... <xn>          (6 casas decimais, separados por um espaço)
MULTIPLAS: SIM|NAO
DEGENERADA: SIM|NAO
ITERACOES: <pivos da Fase 1> <pivos da Fase 2>
ALTERNATIVA: <x1> ... <xn>     (esta linha SO aparece quando MULTIPLAS: SIM)
```

Se o problema for ilimitado:

```
STATUS: ILIMITADA
```

Se o problema for inviável:

```
STATUS: INVIAVEL
```

Regras de formatação:

- Todos os números reais com **exatamente 6 casas decimais** (`printf("%.6f", v)`).
- `ITERACOES` conta apenas os **pivoteamentos das iterações do Simplex** em cada fase; os pivoteamentos eventualmente feitos na transição (Seção 3.4) e o pivoteamento extra da solução alternativa (Seção 3.6) **não** são contados.
- Nenhuma outra mensagem deve ser impressa em `stdout` no modo sem `-t`. Mensagens de depuração, se houver, vão para `stderr`.

6. Modo passo a passo (`-t`)

Com o argumento `-t`, o programa imprime, **antes** do bloco canônico:

- o cabeçalho de cada fase;
- o **tableau completo** no início de cada fase e após **cada** iteração, com uma coluna por variável (na ordem do Passo 3), a coluna `RHS`, uma linha por restrição rotulada com a variável básica e a linha `Z`;
- para cada iteração: qual variável **entra**, qual **sai** e o **elemento pivô**;
- o valor de `w` ao término da Fase 1.

O layout visual é **livre** (larguras de coluna, casas decimais, títulos), desde que todos os elementos acima estejam presentes e legíveis. O exemplo abaixo, produzido pela implementação de referência para o Problema E, é apenas ilustrativo — e reproduz exatamente os tableaus construídos no quadro na aula 5:

=== FASE 1 ===

--- Tableau inicial (Fase 1) ---

BASE	x1	x2	s1	s2	e3	a3	RHS
s1	1.5000	2.0000	1.0000	0.0000	0.0000	0.0000	300.0000
s2	0.5000	1.0000	0.0000	1.0000	0.0000	0.0000	125.0000
a3	1.0000	0.0000	0.0000	0.0000	-1.0000	1.0000	20.0000
Z	-1.0000	0.0000	0.0000	0.0000	1.0000	0.0000	-20.0000

Entra: x1 | Sai: a3 | Pivo: linha 3, coluna x1 (valor 1.0000)

--- Tableau apos iteracao 1 (Fase 1) ---

BASE	x1	x2	s1	s2	e3	a3	RHS
s1	0.0000	2.0000	1.0000	0.0000	1.5000	-1.5000	270.0000
s2	0.0000	1.0000	0.0000	1.0000	0.5000	-0.5000	115.0000
x1	1.0000	0.0000	0.0000	0.0000	-1.0000	1.0000	20.0000
Z	0.0000	0.0000	0.0000	0.0000	0.0000	1.0000	0.0000

Fase 1 concluida: w = 0.000000

=== FASE 2 ===

--- Tableau inicial (Fase 2) ---

BASE	x1	x2	s1	s2	e3	RHS
s1	0.0000	2.0000	1.0000	0.0000	1.5000	270.0000
s2	0.0000	1.0000	0.0000	1.0000	0.5000	115.0000
x1	1.0000	0.0000	0.0000	0.0000	-1.0000	20.0000
Z	0.0000	-25.0000	0.0000	0.0000	-18.0000	360.0000

Entra: x2 | Sai: s2 | Pivo: linha 2, coluna x2 (valor 1.0000)

--- Tableau apos iteracao 1 (Fase 2) ---

BASE	x1	x2	s1	s2	e3	RHS
s1	0.0000	0.0000	1.0000	-2.0000	0.5000	40.0000
x2	0.0000	1.0000	0.0000	1.0000	0.5000	115.0000
x1	1.0000	0.0000	0.0000	0.0000	-1.0000	20.0000
Z	0.0000	0.0000	0.0000	25.0000	-5.5000	3235.0000

Entra: e3 | Sai: s1 | Pivo: linha 1, coluna e3 (valor 0.5000)

--- Tableau apos iteracao 2 (Fase 2) ---

BASE	x1	x2	s1	s2	e3	RHS
e3	0.0000	0.0000	2.0000	-4.0000	1.0000	80.0000
x2	0.0000	1.0000	-1.0000	3.0000	0.0000	75.0000
x1	1.0000	0.0000	2.0000	-4.0000	0.0000	100.0000
Z	0.0000	0.0000	11.0000	3.0000	0.0000	3675.0000

STATUS: OTIMA

Z: 3675.000000

X: 100.000000 75.000000

MULTIPLAS: NAO

DEGENERADA: NAO

ITERACOES: 1 2

7. Casos de teste obrigatórios

Os 11 casos abaixo **serão usados na correção** (junto de outros não divulgados). Todas as saídas foram geradas pela implementação de referência. Guarde-os em arquivos `caso01.txt ... caso11.txt` e verifique cada um antes de entregar.

Caso 1 — Restrição $\geq$ (Problema E da aula 5)

Entrada

```
2 3
MAX
18 25
1.5 2 <= 300
0.5 1 <= 125
1 0 >= 20
```

Saída esperada

```
STATUS: OTIMA
Z: 3675.000000
X: 100.000000 75.000000
MULTIPLAS: NAO
DEGENERADA: NAO
ITERACOES: 1 2
```

Confere com o resultado obtido em aula pelo Big-M e pelo método gráfico.

Caso 2 — Somente $\leq$ (Fase 1 dispensada, 3 variáveis)

Entrada

```
3 3
MAX
5 4 3
2 3 1 <= 5
4 1 2 <= 11
3 4 2 <= 8
```

Saída esperada

```
STATUS: OTIMA
Z: 13.000000
X: 2.000000 0.000000 1.000000
MULTIPLAS: NAO
DEGENERADA: NAO
ITERACOES: 0 2
```

Caso 3 — Múltiplas soluções ótimas

Entrada

```
2 3
MAX
2 2
1 1 <= 10
1 0 <= 8
0 1 <= 8
```

Saída esperada

```
STATUS: OTIMA
Z: 20.000000
X: 8.000000 2.000000
MULTIPLAS: SIM
DEGENERADA: NAO
ITERACOES: 0 2
ALTERNATIVA: 2.000000 8.000000
```

Os dois extremos da aresta ótima discutida no Bloco 7 da aula 5.

Caso 4 — Degenerescência

Entrada

```
2 2
MAX
3 9
1 4 <= 8
1 2 <= 4
```

Saída esperada

```
STATUS: OTIMA
Z: 18.000000
X: 0.000000 2.000000
MULTIPLAS: NAO
DEGENERADA: SIM
ITERACOES: 0 2
```

A segunda iteração é degenerada: troca a base sem mover o ponto nem alterar Z .

Caso 5 — Solução ilimitada

Entrada

```
2 2
MAX
1 1
1 -1 <= 1
-1 1 <= 10
```

Saída esperada

```
STATUS: ILIMITADA
```

Caso 6 — Problema inviável

Entrada

```
2 2
MAX
1 1
1 1 <= 2
1 1 >= 5
```

Saída esperada

```
STATUS: INVIABEL
```

A Fase 1 termina com $w > 0$.

Caso 7 — Minimização com $=$ e $\geq$

Entrada

```
2 3
MIN
4 1
3 1 = 3
4 3 >= 6
1 2 <= 4
```

Saída esperada

```
STATUS: OTIMA
Z: 3.400000
X: 0.400000 1.800000
MULTIPLAS: NAO
DEGENERADA: NAO
ITERACOES: 2 1
```

Caso 8 — Lado direito negativo

Entrada

```
2 2
MAX
3 2
-1 -1 >= -4
1 1 >= 2
```

Saída esperada

```
STATUS: OTIMA
Z: 12.000000
X: 4.000000 0.000000
MULTIPLAS: NAO
DEGENERADA: NAO
ITERACOES: 1 1
```

A primeira restrição vira $x_1 + x_2 \leq 4$ após a multiplicação por -1 .

Caso 9 — Restrição redundante (artificial básica em zero)

Entrada

```
2 2
MAX
2 3
1 1 = 4
2 2 = 8
```

Saída esperada

```
STATUS: OTIMA
Z: 12.000000
X: 0.000000 4.000000
MULTIPLAS: NAO
DEGENERADA: NAO
ITERACOES: 1 1
```

A segunda restrição é múltipla da primeira: ao fim da Fase 1 sobra uma artificial básica em zero cuja linha é toda nula fora das colunas artificiais — a linha deve ser removida (Seção 3.4).

Caso 10 — Degenerada e com múltiplas soluções

Entrada

```
2 2
MAX
1 1
1 1 <= 5
1 1 >= 5
```

Saída esperada

```
STATUS: OTIMA
Z: 5.000000
X: 5.000000 0.000000
MULTIPLAS: SIM
DEGENERADA: SIM
ITERACOES: 1 1
ALTERNATIVA: 0.000000 5.000000
```

Caso 11 — Aresta ótima ilimitada

Entrada

```
2 1
MAX
0 1
0 1 <= 1
```

Saída esperada

```
STATUS: OTIMA
Z: 1.000000
X: 0.000000 1.000000
MULTIPLAS: SIM
DEGENERADA: NAO
ITERACOES: 0 1
ALTERNATIVA: RAO
```

x_1 é não básica com custo reduzido nulo, mas sua coluna não tem coeficiente positivo: o conjunto ótimo é uma semirreta, não um segmento entre dois vértices.

Tolerância da correção: valores numéricos são comparados com tolerância absoluta de $1e-4$. As palavras-chave (STATUS , SIM / NAO , etc.) devem bater exatamente.

7.1 Bateria completa de testes ([bateria_testes_trabalho1.zip](#))

Os 11 casos acima são o mínimo. Junto deste enunciado é distribuída uma **bateria com 107 casos de teste com gabarito**, organizada em complexidade crescente, com um script que confere a saída do seu programa automaticamente:

```
unzip bateria_testes_trabalho1.zip
cd bateria_testes_trabalho1
python3 verificar.py ./simplex          # roda tudo
python3 verificar.py ./simplex --grupo A # roda só um grupo
```

grupo	casos	o que exercita
A	11	somente $\leq$: leitura, tableau, pivô, Fase 2 (sem Fase 1)
B	7	uma restrição $\geq$: a primeira variável artificial
C	6	várias $\geq$ simultâneas
D	7	igualdades, incluindo restrição redundante
E	22	cada caso especial isolado no menor exemplo possível
F	41	grade completa: tipos de restrição $\times$ propriedade da solução
G	13	escala, de $n=5$ até $n=1600$ (o maior leva ~ 30 s)

Os grupos são feitos para serem resolvidos **em ordem**: use o grupo A enquanto o programa ainda não tem Fase 1, o B assim que ela existir, e assim por diante. O `MANIFESTO.md` de dentro do pacote descreve caso a caso o que está sendo testado, traz a matriz de cobertura completa e explica exatamente o que o verificador compara.

Os casos maiores do grupo G não vêm prontos (passam de 20 MB somados); um script determinístico incluído no pacote os reconstrói:

```
python3 gerador_grandes.py
```

Passar na bateria não é a nota, e a correção usará também casos não divulgados — mas espera-se que o programa entregue passe nela, e o resumo final do verificador deve ser colado no seu `README.md` (Seção 9).

8. Requisitos de implementação

- Linguagem:** C (padrão C11) ou C++ (padrão C++17), à sua escolha. Um único arquivo `simplex.c` ou `simplex.cpp` é aceitável; se preferir dividir em módulos, inclua um `Makefile`.
- Compilação** — o programa deve compilar **sem erros nem warnings** com: `gcc -std=c11 -O2 -Wall -Wextra -o simplex simplex.c -lm` `g++ -std=c++17 -O2 -Wall -Wextra -o simplex simplex.cpp`
- Bibliotecas:** apenas a biblioteca padrão da linguagem. É **proibido** usar bibliotecas de álgebra linear ou qualquer resolvedor pronto de PL (GLPK, lp_solve, CPLEX, Eigen, etc.). A implementação do Simplex deve ser inteiramente sua.
- Alocação:** o tableau deve ser alocado dinamicamente conforme n e m lidos (nada de matriz global de tamanho fixo arbitrário). Em C, libere a memória alocada.
- Robustez:** o programa não deve travar, entrar em laço infinito nem acessar memória inválida em nenhum dos casos de teste. Estabeleça um limite de segurança de iterações (por exemplo, $100 * (n + m)$) e, se ele for atingido, encerre com mensagem em `stderr` — isso indica erro de implementação ou ciclagem.

6. **Organização:** separe o código em funções com responsabilidade clara (por exemplo `ler_entrada`, `montar_tableau`, `escolher_coluna`, `teste_razao`, `pivotear`, `fase1`, `fase2`, `imprimir_tableau`). Comente as passagens não óbvias, especialmente a canonicalização da linha `z` e a transição entre fases.
-

9. O que entregar

Um único arquivo `.zip` nomeado `SEUNOME_trabalho1.zip`, contendo:

1. o(s) arquivo(s) de código-fonte (e o `Makefile`, se houver);
 2. os arquivos `caso01.txt` ... `caso11.txt` da Seção 7;
 3. **pelo menos 2 casos de teste próprios**, criados por você, em arquivos `meuteste01.txt`, `meuteste02.txt`, cada um acompanhado da saída esperada (`meuteste01.out`, ...) e de uma frase explicando o que aquele caso exercita;
 4. um `README.md` curto (1 a 2 páginas) contendo:
 - como compilar e executar;
 - as principais decisões de implementação (estrutura de dados do tableau, como você tratou a transição entre fases, como detectou os casos especiais);
 - **o resumo final do `verificar.py`** (as últimas linhas, com o placar por grupo) colado como bloco de texto, e a lista dos casos que ainda falham, se houver (**seja honesto** — um trabalho parcial bem diagnosticado vale mais que um trabalho que alega funcionar e não funciona);
 - um parágrafo respondendo: *por que o método das Duas Fases é preferível ao Big-M em uma implementação computacional?* e um comentário sobre a limitação apontada na Seção 3.6 (degenerescência × múltiplas soluções).
-

10. Critérios de avaliação (nota de 0 a 25)

A nota **deste trabalho** vai de 0 a 25, conforme a rubrica abaixo. Essa nota não é somada diretamente à nota final da disciplina: ela entra na **média das tarefas do Bloco 1**, e é essa média que vale os 25 pontos indicados no cabeçalho. Em outras palavras, os 25 desta seção são a **escala** desta tarefa, não uma segunda parcela de 25 pontos.

#	Item	Pontos
1	Leitura e padronização: leitura correta da entrada, tratamento de <code>MIN</code> , de <code>b < 0</code> , criação e ordenação correta de folgas, excessos e artificiais, base inicial	4
2	Fase 1: construção canônica da linha <code>Z</code> , iterações corretas, proibição de reentrada de artificiais, detecção de inviabilidade, tratamento da transição (artificial básica em zero e linha redundante)	5
3	Fase 2: descarte das artificiais, recuperação e canonicalização do objetivo original, iterações até a otimalidade, valor de <code>Z</code> correto no sentido original do problema	5
4	Casos especiais: ilimitada, múltiplas soluções + solução alternativa (incluindo <code>RAIO</code>), degenerescência	5
5	Modo <code>-t</code>: tableaux por iteração, variável que entra/sai, elemento pivô, valor de <code>w</code> ao fim da Fase 1	3
6	Qualidade: organização e legibilidade do código, compilação limpa, <code>README.md</code> com o resumo da bateria (Seção 7.1), casos de teste próprios	3
	Total	25

Penalidades: −2 pontos por formato de saída divergente da Seção 5 (linhas fora de ordem, casas decimais erradas, texto extra em `stdout`); −2 pontos por warnings de compilação; até −5 pontos por ausência do `README.md`. Entregas com atraso serão avaliadas caso a caso.

11. Roteiro sugerido de desenvolvimento

Não tente escrever tudo de uma vez. Uma ordem que funciona bem:

1. **Leitura + impressão do tableau inicial.** Antes de qualquer iteração, imprima o tableau montado e confira à mão com o Caso 1. Se a padronização estiver errada, todo o resto estará.
2. **Um pivoteamento.** Implemente `pivotear(linha, coluna)` e teste-o isoladamente, conferindo com o primeiro tableau da aula 5.
3. **Fase 2 apenas,** com o Caso 2 (só `<=`, sem artificiais). Aqui você já resolve um PL completo — e o **grupo A** inteiro da bateria já deve passar.
4. **Fase 1 + transição,** com os Casos 1, 6, 7 e 9, e depois os **grupos B, C e D**.
5. **Casos especiais,** com os Casos 3, 4, 5, 10 e 11, e depois os **grupos E e F**.
6. **Modo `-t`,** limpeza do código e, por último, o **grupo G** (desempenho).

Uma dica de depuração: compare a saída do seu programa com um resolvidor de referência (por exemplo, `scipy.optimize.linprog` em Python, ou o Solver do LibreOffice Calc) — **apenas para conferir o valor ótimo**, jamais como parte da entrega.

12. Bônus opcionais (até 3 pontos extras na nota desta tarefa)

Os pontos de bônus compensam perdas na rubrica da Seção 10, respeitado o teto de 25 na nota desta tarefa — não é possível ultrapassá-lo.

Escolha no máximo dois, descreva-os no `README.md` e ative-os por argumentos de linha de comando adicionais:

- **-bigm** — resolver o mesmo problema pelo método Big-M e comparar, no `-t`, o número de iterações com o das Duas Fases nos 11 casos.
 - **-bland** — implementar a Regra de Bland (entra a variável de **menor índice** com custo reduzido negativo; sai a de menor índice em caso de empate) e discutir, no `README.md`, seu efeito sobre a ciclagem e sobre o número de iterações.
 - **-frac** — usar aritmética exata com frações (numerador/denominador inteiros com `long` e simplificação por MDC) em vez de `double`, e comentar as diferenças observadas.
 - **-dual** — imprimir os valores das variáveis duais (preços-sombra) lidos na linha `Z` da tabela ótima, nas colunas das folgas — uma antecipação do conteúdo de dualidade.
-

13. Integridade acadêmica

O trabalho é **individual**. Discutir ideias com colegas é permitido e saudável; compartilhar ou copiar código, não. Trabalhos com trechos de código idênticos (entre colegas ou com fontes externas) serão zerados para todos os envolvidos.

O uso de assistentes de IA é permitido **como ferramenta de estudo e depuração**, mas o código entregue deve ter sido digitado por você e ser compreendido integralmente por você: qualquer aluno poderá ser convidado a explicar oralmente qualquer trecho do próprio código, e a nota final poderá ser ajustada com base nessa arguição. Se usou IA, registre no `README.md` em que partes e de que forma — isso não reduz a nota; omitir, sim.

14. Erros mais comuns (leia antes de entregar)

- **Esquecer de canonicalizar a linha `Z` no início da Fase 2.** A base herdada da Fase 1 quase nunca tem custo reduzido zero sob o objetivo original — sem esse passo, a resposta sai errada mesmo com todo o resto correto.
 - **Aceitar coeficiente pivô negativo ou nulo no teste da razão.** A regra exige `a_ip > 0` **estritamente**; ignorar isso costuma produzir "soluções" que violam restrições.
 - **Deixar artificiais reentrarem na base durante a Fase 1.**
 - **Confundir `w` com `-w`** ao testar a viabilidade ao fim da Fase 1.
 - **Esquecer de inverter o sinal de `Z`** ao reportar problemas de minimização.
 - **Comparar `double` com `== 0`.** Use sempre a tolerância.
 - **Imprimir texto extra em `stdout`** ("Digite a entrada:", "Resultado:") — isso quebra a correção automática. Prompts, se existirem, vão para `stderr`.
-

Dúvidas sobre o enunciado devem ser postadas no mural do Google Classroom, para que a resposta fique visível a toda a turma.